

KernelAscent

A benchmark for recursive self-improvement in GPU-kernel optimization,
and what it taught us about measuring self-improvement at all

KernelAscent

September 26, 2026

Abstract

KernelAscent asks whether a model can make itself better at writing GPU kernels, and whether that improvement compounds. We built a five-rung ladder — one-shot capability, weight-level self-training, frozen-weight procedure self-editing, a closed model rewriting an open trainee’s harness, and self-play in which a model authors the frontier it must then beat — and ran it across scales from 0.5B to 32B.

Our primary finding is about the measurement rather than the models. In a controlled comparison where two cells differ in one environment variable and nothing else, the benchmark’s original scoring rule reports a contrast of -0.0002 across ten rounds while the identical generations, rescored, report $+0.436$. Best-of- k scoring cannot distinguish a model that solves a task once in k from one that solves it k times in k , which is precisely the quantity self-training moves; and because a matched control arm reaches the saturation point within three or four rounds, the blindness is scheduled by the design rather than risked by it.

The ladder’s upper rungs did not return nulls. They returned undefined quantities, for one shared reason that is computable in advance: the self-referential loop received almost no input. The weight rung’s learner received under one verified example per round with over 40% of rounds empty; the procedure rung’s archive grew by four entries across six rounds, or none; the self-play author had 4 of 173 proposals accepted. We therefore make no claim that recursive self-improvement fails here, and we argue instead that a self-improvement benchmark must demonstrate its loop has throughput before any contrast it reports carries information.

We report what survives with its scope attached, and Section A lists every claim this project has retracted, narrowed, or excluded, each with the evidence that forced the change.

Contents

1	Why this benchmark exists, and what it found	3
1.1	The same runs, two scorers, opposite conclusions	3
1.2	The blindness is scheduled, not accidental	3
1.3	Three rungs did not produce nulls; they produced undefined measurements	3
1.4	Feeding the loop: what the throughput precondition buys	4
1.5	A non-LLM baseline that scores below parity	4
1.6	What the reader should take from the rest of this report	5
2	The benchmark	6
2.1	The task	6
2.2	Scoring	6
2.3	The recurrence under test	6

2.4	The five rungs	7
2.5	Preconditions, and why they are stated before the results	7
3	Related work	8
3.1	Kernel generation as an evaluation substrate	8
3.2	Self-improvement and self-modification	8
3.3	Why the instrument work is the contribution	9
4	Infrastructure: three weeks on 72 GPUs	10
5	Results by rung	11
5.1	T1 — Capability	14
5.2	T2 — Weight-RSI: a rigorously-powered compounding null	14
5.3	T3 — Procedure-RSI	16
5.4	T4 — Closed→Open	17
5.5	T5 — Self-play: an author-yield failure, not a measured null	17
6	Cross-cutting analyses: the mechanism	18
6.1	Coverage vs. scale, and the “correctness wall” as a sampling artifact	18
6.2	Zero-score forensics: the wall is a formation failure	19
6.3	Search beats training at matched compute	19
6.4	WHY-RSI: the weight-level mechanism and the inverted-U in scale	19
6.5	Probe-as-intervention: correctness is represented better than it is generated	20
7	Instrument validity: how a self-improvement benchmark manufactures a null	21
8	Synthesis: why self-improvement does not compound here	27
8.1	The discovery-to-assimilation bottleneck	27
9	The accumulated intuition (what three weeks taught us)	28
10	What would break the plateau: predictions	29
11	Limitations and scope	29
12	Conclusion	30
A	What was removed, narrowed, or retracted — and why	33
A.1	Retracted: claims the data does not support	33
A.2	Narrowed: claims that hold with a smaller scope	34
A.3	Excluded: data that cannot answer the question it was collected for	34
A.4	Provisional: retained but not load-bearing	34
A.5	Known confounds carried into the results	35
B	Probe-as-intervention, analysed as a secondary result	36

1 Why this benchmark exists, and what it found

A self-improvement benchmark reports a number. The number is only worth reading if the measurement could have moved. This section states the four results that make the rest of the report worth reading, each of which is about whether the instrument can see what it claims to measure.

1.1 The same runs, two scorers, opposite conclusions

The central result is a controlled comparison of two scoring rules on one experiment. Two cells differ in a single environment variable and in nothing else — same base model, same seed, same sampling budget k , same task bank, same three arms, same code path. One scores each round with the headroom-normalised best-of- k rule the benchmark was designed around; the other scores the identical generations by pass rate, the fraction of k candidates that verify.

Across the ten rounds in which both the lineage and reset arms have reached correct-at-parity, the headroom contrast reports a mean of -0.0002 and never departs from zero by more than 0.004 . The same ten rounds report a mean of $+0.436$ under pass rate, with a minimum of $+0.360$. The two ranges are separated by a factor of roughly ninety and do not approach one another.

This is not a claim that pass rate is the correct metric; it has a ceiling of its own, reached in three of four cells. It is a demonstration that a benchmark can return a confident null on a difference its own runs contain. Table 5 is the evidence, and it requires no statistics to read.

1.2 The blindness is scheduled, not accidental

Best-of- k is identical for a model that solves a task one time in k and one that solves it k times in k . Self-training acts on exactly that quantity. The consequence is visible in the arm levels rather than in any summary statistic. Partitioning every round of all four headroom cells, across both model scales, by whether the two arms are separated at all: the rounds in which they sit within 0.02 of each other average essentially zero and never depart from it by more than 0.004 , while the rounds with a gap average two orders of magnitude larger. The counts and the two means are computed from the artifacts and printed in the caption of Table 5; they move as cells add rounds, and the separation has held across every round collected so far.

The contrast is therefore measuring the gap between the arms and nothing else, faithfully, at all times. Best-of- k compresses both arms onto correct-at-parity and holds them there, so the gap it faithfully reports is zero. It recovers the moment a gap reopens, and the caption names the clearest instance the data contains.

The design consequence is the part worth carrying: a matched reset learner reaches parity within three or four rounds, so the measurement’s useful lifetime is fixed by the *control* arm rather than by the treatment. This cannot be repaired by choosing a better treatment or running longer. A well-behaved control ends the measurement by behaving well.

1.3 Three rungs did not produce nulls; they produced undefined measurements

Three of the five rungs failed for one shared reason, and it is computable before a run rather than diagnosed after. Each rung was built around a different channel, in a different lab, and in each the channel received almost no input.

Table 1: The self-referential loop’s throughput, measured. Each rung was designed to test whether a specific channel compounds; each channel was starved. The T2 figure is recomputed from the 30 rounds of the registered primary that are admissible under Amendments 5 and 6.

Rung	What feeds the loop	Measured
T2 (weight-RSI)	verified self-generated training examples	0.93 per round, zero in 43% of rounds
T3 (procedure-RSI)	kernels entering the archive after grading	4 across six rounds, or 0
T5 (self-play)	accepted model-authored tasks	4 of 173 proposed (2.3%)

The arithmetic was available in advance. Expected training examples per round is $n_{\text{train}} \times k \times \text{yield}$. At the registered configuration this is $3 \times 10 \times 0.032 = 0.96$, and the artifacts agree to two decimals (Table 1 is recomputed at build time and moves as rounds land). We ran 57 trajectories and 228 rounds of a loop that could not compound, and the calculation that says so is one line.

The claim this supports is not that recursive self-improvement fails. It is that **a self-improvement benchmark must demonstrate that its loop has throughput before any contrast it reports carries information**, and that this is a precondition to check rather than a result to discover.

1.4 Feeding the loop: what the throughput precondition buys

The starvation result is a claim about a precondition, so the obvious test is to meet it. The **fed** cells run the same lab, the same three arms and the same contrast as the registered primary, changing one thing: the learner is given tasks from a verified kernel bank at `--n-train 35` rather than the registered 3. Table 5 reports both trajectories in full.

The result that carries weight is the one the precondition predicts and which needs no statistics. Mean verified examples per round go from 0.96 to 50.8 and 27.4, and the per-round count *grows within each run* — 6 to 104 in one, 7 to 84 in the other. Each round’s model verifies more of what it generates, so the next round trains on more. That is what a self-improvement loop is supposed to do, and it is the first time this benchmark has observed it in any cell.

The contrast itself we report and do not interpret. Both trajectories are positive, the trajectory-level mean is +0.088, and both clear the registered magnitude threshold on their last two rounds. They are also $n = 2$ against a registered minimum of three seeds, so no interval is quotable and the registered decision rule cannot be evaluated. We state the numbers, we state that they do not meet the bar we set in advance, and we do not round the difference in our favour. The honest summary is that feeding the loop made the contrast *measurable*, which is a claim about the instrument and is settled, and that whether it makes the contrast *positive* is a claim about models that these two trajectories cannot carry.

1.5 A non-LLM baseline that scores below parity

The most common objection to results in this domain is that a model reaching correct-at-parity is simply failing. We answered it by submitting a non-LLM tool to the benchmark on its own terms. `torch.compile(mode="max-autotune")`, scored exactly as a model candidate is, reaches a median of 0.408 — below correct-at-parity — because it is slower than default `torch.compile` on 27 of 29 tasks. A model matching the compiled baseline is therefore outperforming production autotuning on this bank, not failing to beat a weak baseline.

1.6 What the reader should take from the rest of this report

The four results above are about the instrument. The remainder of the report is the ladder that produced them: five rungs, their designs, their controls, and every number with its uncertainty and its scope. Section A lists every result that was removed or narrowed during the work, with the evidence that forced each change, because a report that only shows its surviving claims has hidden the part a reader most needs in order to trust the rest.

2 The benchmark

2.1 The task

A task is a self-contained PyTorch module with a fixed dtype, a shape, and an input generator. A submission replaces its `forward` with a faster implementation that produces the same output. Correctness is graded against an **fp32 gold** model built from the same weights, with a per-task tolerance derived from the reference’s own dtype error rather than a global epsilon,

$$\text{bound} = \max(2 \times 10^{-2}, 2\varepsilon_{\text{ref}}),$$

so a task whose reference is itself numerically loose does not silently accept a wrong kernel. Every candidate executes in an isolated subprocess: a malformed kernel can crash, hang, or corrupt a CUDA context without affecting the trainer that produced it.

2.2 Scoring

Speed is normalised per task against a roofline [23] ceiling rather than a fixed anchor. With s the candidate’s speedup over the `torch.compile` [1] baseline and c the achievable ceiling,

$$c = \text{clip}\left(\frac{t_{\text{compiled}}}{t_{\text{roofline}}}, 1.1, 50\right), \quad t_{\text{roofline}} = \max\left(\frac{\text{FLOPs}}{\text{peak}}, \frac{\text{bytes}}{\text{bandwidth}}\right),$$

$$\text{score}(s) = 0.5 + 0.5 \text{clip}\left(\frac{s-1}{c-1}, 0, 1\right).$$

A score of 0.5 is correct-at-parity with the baseline; 1.0 is the physically achievable ceiling for that task. Normalising by the roofline makes a task with $1.1\times$ of available slack and a task with $40\times$ contribute equally to a mean, which a fixed anchor does not.

The second scorer. Pass rate, $\hat{p} = n_{\text{ok}}/k$, is the fraction of the k sampled candidates that verify. It is an unbiased estimator of the per-sample success probability and, unlike best-of- k , its sensitivity does not vanish as that probability rises. It was declared post-hoc in a pre-registration amendment with its falsifier fixed in advance, is reported as its own experiment set, and is **never pooled with** the headroom boards. Section 1.1 is the reason both exist.

2.3 The recurrence under test

Write $\theta(t)$ for the adapter weights after round t , $\pi(\theta)$ for the sampling distribution they induce, $D(t)$ for the kernels the model generated and the grader verified in round t , and $C(t)$ for held-out capability. A round is

$$\theta(t+1) = T(\theta(t), D(t)), \quad D(t) \sim \pi(\theta(t)).$$

Compounding is a claim about the derivative of that map: this round’s gain grows with the capability already reached, $\partial C(t+1)/\partial C(t) > 0$. The claim is trivially confoundable with “the model sampled more”, so every rung is a *differenced* design in which the contrast isolates one causal channel.

2.4 The five rungs

T1 — Capability.

One-shot kernel authoring at a fixed budget. No recursion. Establishes whether the task is reachable at all, which is a precondition for every rung above it, and separates *attempting* a kernel from *verifying* one.

T2 — Weight-RSI.

Three arms at matched compute. **Lineage** LoRA [7]-trains on its own verified kernels and carries weights forward. **Reset** re-initialises every round and trains only on that round’s producer data. **Best-of- N** never trains and samples the frozen base at the cumulative budget $k(r+1)$. Both learners see byte-identical per-round data, so lineage – reset isolates weight *accumulation*, and lineage – best-of- N separates training from search at equal samples.

T3 — Procedure-RSI.

The model rewrites its own executable strategy library and keeps an archive of verified kernels, with weights frozen. The registered signal F_g is the amount by which a newer self-written procedure beats the older one on held-out tasks, which is a producer-quality contrast rather than a capability curve.

T4 — Closed→Open.

A closed frontier model rewrites the training harness of an open trainee. Improvement, if any, transfers through tooling rather than through weights.

T5 — Self-play.

Three arms that differ only in who writes the tasks. **S** holds the frontier fixed; **F** escalates it with tasks written by the frozen base; **L** escalates it with tasks written by the evolving model. $L - S$ is the total curriculum effect, $F - S$ the effect with no author update, and $L - F$ the cross term between author quality and solver quality — the only self-referential term in the ladder. Every authored task passes a gate rejecting constant-output, identity, no-op and trivial-runtime proposals, and its provenance is logged as model-proposed or programmatic backstop.

2.5 Preconditions, and why they are stated before the results

Two quantities determine whether any rung’s contrast can carry information, and both are computable before a run.

Reachability. If the frozen base cannot clear the scoring anchor on a task, that task contributes no signal to any arm. T1 measures this directly.

Throughput. The self-referential loop must receive input. Expected training examples per round is $n_{\text{train}} \times k \times \text{yield}$, and the analogous quantity for T3 is archive growth and for T5 is accepted authored tasks. Section 1.3 reports what these were, and they are the reason three rungs report undefined quantities rather than nulls.

3 Related work

3.1 Kernel generation as an evaluation substrate

KernelBench [15] established the substrate this benchmark builds on: a model is given a PyTorch module and asked for a faster implementation, and the submission is graded by execution rather than by resemblance to a reference. KernelAscent inherits that task and changes the question. KernelBench asks whether a model *can* write an efficient kernel; we ask whether that ability *compounds* when the model trains on its own verified output. The distinction matters for design rather than framing: a one-shot benchmark is free to use tasks a strong model solves at parity, whereas a compounding benchmark is not, because a task the control arm already solves contributes nothing to a contrast between arms (Section 7).

Subsequent work has strengthened the grading contract along axes we depend on and do not contribute to. KernelBench-Verified [9] adds stronger input-distribution checks and realistic precision settings; its authors are explicit that finite hidden tests are not a proof of general correctness, and the same caveat applies to our grader. SOL-ExecBench [14] evaluates against modelled hardware limits, which is the same motivation as our per-task roofline ceiling [23], though we use the ceiling to normalise a score across heterogeneous tasks rather than to certify optimality. FlashInferBench [3] and its trace schema [4] move in the opposite direction from us, toward real inference workloads and runtime substitution of generated kernels; that is the deployment question, and a benchmark that cannot yet show its own measurement has range is not in a position to ask it.

At the system level, MLPerf Inference [12] and vLLM’s serving benchmarks [21] measure end-to-end throughput and latency under declared quality constraints. Our unit is different: we measure the change a self-improving agent produces in its own kernel-writing ability, not the performance of a serving stack, and we make no claim of MLPerf compliance.

3.2 Self-improvement and self-modification

The recursive self-improvement line this benchmark is aimed at spans several distinct mechanisms, and the five rungs of Section 2 are an attempt to separate them rather than to rank them. Voyager [22] accumulates an executable skill library, which is improvement carried in *stored artifacts* and is closest to our T3 procedure rung. STOP [19] recursively optimises the scaffolding around a code generator, and the Darwin Gödel Machine [16] maintains an archive of self-modifying variants; both improve the *procedure* while leaving weights fixed. HyperAgents [26] defines *improvement@k* and a transfer experiment, which is the prior evaluation closest in spirit to the contrasts reported here. Metaⁿ [10] formulates recursion at the meta-agent level; we cite it for positioning and do not independently verify its empirical claims. AlphaEvolve [5] is the existence proof that automated optimisation can reach deployed computing systems, which is what makes the domain worth measuring carefully.

Our weight rung uses verified rejection-sampling fine-tuning, the method introduced by STaR [25] and scaled by ReST [6]: sample, keep what verifies, train on it, repeat. The contribution here is not the method but the controls around it. A rising curve under this procedure is consistent with a better producer, with more data, and with more sampling, and our three-arm decomposition exists to separate those.

The regularized objective in Section 1.4 adopts the KL-to-reference-policy penalty standard in RLHF [27], using the low-variance k3 estimator [18]. The novelty is the use rather than the term: our mechanism analysis finds that sustained adapter drift and round-0 retention discriminate the runs that improve, but finds it *post hoc*, as properties of runs that already happened. A trust region makes both of them settings.

3.3 Why the instrument work is the contribution

Two lines of work motivate the emphasis of this report. METR [11] documents frontier models manipulating tests and scoring mechanisms in automated research settings, which is the case for grading a submission independently of the agent that produced it. We report a failure that is adjacent but distinct, and in our view more common: nobody gamed anything here, and the measurement still failed, because a scoring rule that cannot resolve the quantity under study returns a confident null rather than an error. The infrastructure literature on verification and isolation — compute sanitizers [13], container security [2] — addresses whether a harness can be trusted to run untrusted code. Our finding is that a harness can be entirely trustworthy in that sense, execute every candidate correctly, and still be incapable of measuring what the benchmark was built to measure.

To our knowledge no prior self-improvement benchmark reports a dynamic-range precondition, a throughput precondition, or a ledger of its own withdrawn results. We think all three should be standard, and Section 7 argues the case from our own failures rather than from principle.

4 Infrastructure: three weeks on 72 GPUs

The experiments ran on a 9-node, 72-GPU cluster (A100-class), continuously for three weeks. A few infrastructure decisions shaped what was learnable, and several operational lessons are worth recording because they materially affected the results.

Compute discipline. Every rung’s recursive arm is compute-matched to its control by a full ledger that counts generation *and* verification *and* training tokens/steps. Without this, “self-training helped” is unfalsifiable — it may just be more sampling. The matched ledger is what turns T2 from a demo into a test.

Resumability. All runs checkpoint to S3 on a 300s daemon so an instance death never loses more than one interval, and pods push independently of the operator’s credentials. This is what made a three-week run survive tunnel drops, credential expiry, and node churn.

Operational lessons (recorded because they bit us).

- **Grading dominates wall-clock.** Crash-isolated per-kernel grading (each candidate graded in its own process so a segfaulting kernel cannot take down the harness) costs ~ 12 s/kernel. For coverage-style experiments requiring hundreds of grades per evaluation, a single round can exceed 2.5 hours — which made the randomized coverage-transplant experiment *computationally infeasible* and forced us to establish the coverage mechanism more cheaply from p -maps and forensics instead.
- **Large-model rounds are slow.** A 7B compounding round costs ~ 64 minutes. At six rounds per seed that is ~ 6.4 hours per seed, so the large-scale arms accumulate statistical power slowly and are sensitive to any run reset from node churn — the reason our 7B/14B equivalence is pooled-significant but not yet individually powered.
- **Measurement hygiene is a result-integrity issue.** We twice caught inflated counts caused by stale local caches masking a silently-failed sync, and once caught a “fleet collapse” that was merely a dropped port-forward tunnel hiding two dozen busy GPUs. Every headline number in this report is taken from a *verified* re-read of canonical storage, not a convenience cache.
- **A single silent bug can fabricate a finding.** An early “compounding collapse” (both train and held-out C dropping to exactly 0.000 after one SFT step) was a generation-after-SFT bug, not a scientific null; a zero-update adapter evaluated correctly, which is how we distinguished the two. Pre-fix runs are therefore *uninterpretable* and are excluded from every statistic by a canonical-seed allowlist.

Intuition. *The most dangerous failure mode in an autonomous, long-horizon run is not a crash — it is a plausible wrong number. Crashes announce themselves; a stale cache or a contaminated seed quietly shifts an estimate. The single highest-leverage habit of the whole run was refusing to trust any number that had not been re-derived from canonical storage with the broken seeds excluded.*

5 Results by rung

Every table in this section is regenerated from stored artifacts at build time. A cell that has not reached its registered depth is reported as incomplete rather than averaged in, and a table whose inputs cannot support it is omitted with a stated reason rather than filled. Where the narrative below quotes a figure, the table is authoritative.

Table 2: T1-kernel. Solve-rate alone is a compliance metric: a submission that ignores the kernel instruction and rewrites the reference in torch verifies trivially. Reported instead as a cascade. [†] marks an incomplete cell. Generation budgets differ: the 0.5B–14B cells ran at $k=12$ and 32B at $k=6$, so 32B contributes half the candidates. The reported quantity is a *rate* and is budget-independent in expectation, but 32B’s estimate is correspondingly noisier. The curve is a **U**: it falls from 0.5B to a minimum of 0 at 14B and recovers to 3.0% at 32B (Fisher $p=0.015$ against 14B). Extraction policy: **strict**. Under **strict** the extractor discards 66% of 0.5B generations and 5% of 14B generations, so any trend across scale must be shown under both policies before it is attributed to the models.

Scale	Generations	Parsed	Attempted kernel	Verified	Kernel-verified
0.5B	348	118 (34%)	74 (63%)	11	4 (5.4%)
1.5B	348	132 (38%)	74 (56%)	11	3 (4.1%)
3B	348	253 (73%)	184 (73%)	9	6 (3.3%)
7B	348	335 (96%)	262 (78%)	5	4 (1.5%)
14B	348	331 (95%)	219 (66%)	0	0 (0.0%)
32B	174	173 (99%)	167 (97%)	5	5 (3.0%)

Table 3: T1-kernel robustness to extraction policy. Verify-given-attempt under KA **EXTRACT=strict** and **lenient**; identical tasks, k , and token budget, differing only in how a generation is converted to a candidate. The endpoint contrast (0.5B vs 14B) holds under both policies and strengthens under **lenient**. Note that 14B is the curve’s *minimum* and not its largest scale: the completed 32B sweep recovers to 3.0% (Fisher $p=0.015$ against 14B), so this is a contrast with the floor. The monotone ordering does not: it is an artifact of a filter whose severity correlates with scale.

Scale	strict			lenient		
	Attempts	Verified	Rate	Attempts	Verified	Rate
0.5B	74	4	5.41%	254	24	9.45%
1.5B	74	3	4.05%	237	7	2.95%
3B	184	6	3.26%	328	26	7.93%
7B	262	4	1.53%	344	10	2.91%
14B	219	0	0.00%	338	2	0.59%

Table 4: 14B verify-given-attempt, replicated across independent routes. Two runs differing in route, sampling budget k , and teacher bank. The model parses as a kernel attempt in 98% of its candidates and almost never produces one that verifies, so the result is not an artifact of extraction, prompt compliance, or a single unlucky run.

Run	Route	k	Attempted	Kernel-verified
t1kl.q14	T1 (lenient)	12	338	2 (0.59%)
r4_kernel.q14	R4	6	171	1 (0.58%)

Table 5: A non-LLM baseline on the benchmark’s own terms: `torch.compile(mode="max-autotune")` scored exactly as a model submission is (speedup over the compiled baseline, same per-task roofline ceiling, same timing harness). It lands *below* correct-at-parity because it is slower than default `torch.compile` on 27 of 29 tasks. A model scoring 0.50 therefore beats production autotuning here. Reported per tier because compile gain over eager differs by $\sim 12\times$ between L1 and L2.

Tier	Tasks	Median score	Median speedup vs compiled
L1	10	0.381	0.611
L2	15	0.408	0.868
L3	4	0.505	1.063
All	29	0.408	0.865

Table 6: T2 weight-RSI under pass rate (n_{ok}/k), the `h100/passrate` set. Reported trajectory-level: one value per cell, the mean of its last two rounds, since rounds inside a cell share an adapter. Lineage beats a matched reset learner that sees identical per-round data, which isolates weight accumulation. Teacher injection changes it by +0.005. **No interval is quoted** at $n=2$ trajectories per arm. C_{lineage} reaches the metric’s ceiling of 1.0 in three of four cells, so the final rounds measure the bound rather than the learner. Per PREREGISTRATION Amendment 1 this board is never pooled with, or compared against, any headroom result.

Cell	Arm	Rounds	lineage – reset	final C_{lineage}
control.q15.s1	control	8	+0.450	0.88
control.q15.s2	control	8	+0.430	0.98
inject.q15.s1	inject	8	+0.500	1.00
inject.q15.s2	inject	8	+0.390	1.00
control, trajectory mean			+0.440	
inject, trajectory mean			+0.445	

Table 7: The same experiment under two scorers. `t2kc` and `t2kp` differ in `KA_SCORE` alone: same model, seed, k , lab, task bank and arms. $\dagger$ marks a round where *both* arms have reached correct-at-parity (≥ 0.49). In all 10 such rounds the headroom contrast is within 0.004 of zero (mean -0.0002), while the same rounds report a mean of $+0.436$ under pass rate. The contrast measures the *gap* between the arms and nothing else: across all four headroom cells and both scales, the 18 rounds with arms within 0.02 of each other average -0.0003 and never exceed 0.004, while the 14 rounds with a gap average $+0.144$. It is not broken by saturation and recovers when a gap reopens (`q3_s2` reads $+0.002$ at 0.50/0.50 then $+0.201$ at 0.50/0.30). The difficulty is that best-of- k compresses both arms onto correct-at-parity and holds them there, and a matched reset learner arrives within three or four rounds, so the measurement’s useful lifetime is fixed by the control arm rather than by the treatment. The two boards are never *pooled* (Amendment 1); they are placed side by side here to compare the scorers, not the results.

Cell	Round	$C_{\text{lin}} / C_{\text{reset}}$	headroom	pass rate
q15_s1	0	0.20 / 0.10	+0.101	+0.020
q15_s1	1	0.40 / 0.20	+0.200	+0.080
q15_s1	2	0.51 / 0.20	+0.304	+0.220
q15_s1	3	0.50 / 0.51	-0.004 $\dagger$	+0.440
q15_s1	4	0.50 / 0.50	+0.000 $\dagger$	+0.520
q15_s1	5	0.50 / 0.50	-0.000 $\dagger$	+0.380
q15_s1	6	0.50 / 0.50	+0.001 $\dagger$	+0.380
q15_s1	7	0.50 / 0.50	-0.001 $\dagger$	+0.520
q15_s2	0	0.00 / 0.20	-0.201	+0.020
q15_s2	1	0.29 / 0.10	+0.189	-0.020
q15_s2	2	0.30 / 0.30	-0.004	+0.080
q15_s2	3	0.50 / 0.50	+0.001 $\dagger$	+0.380
q15_s2	4	0.50 / 0.50	-0.000 $\dagger$	+0.360
q15_s2	5	0.50 / 0.50	+0.001 $\dagger$	+0.520
q15_s2	6	0.50 / 0.50	-0.000 $\dagger$	+0.400
q15_s2	7	0.50 / 0.50	+0.000 $\dagger$	+0.460

Table 8: The same lab and the same contrast, with the learner fed. These cells run `--n-train 35` against a verified kernel bank; the registered primary runs `--n-train 3`, which the throughput arithmetic puts at 0.96 examples per round. Mean n_{ex} here is 50.8 and 27.4, so both trajectories clear Amendment 6’s floor of two by more than a factor of ten, and both carry `adapter_restored`. Every round of both is reported. **No interval is quoted and no effect is claimed:** $n = 2$ trajectories against a registered minimum of three seeds, and a 95% interval on a two-point mean would assert precision the design cannot support. The trajectory-level mean is $+0.088$. The column that carries weight is n_{ex} , which grows within both runs (6 to 104 and 7 to 84) and is the input the contrast lacked everywhere else in this report.

Cell	Rounds	mean n_{ex}	<code>self-fresh</code> per round					mean	last 2
q15_s1	5	50.8	+0.074	+0.132	+0.206	+0.130	+0.089	+0.126	+0.110
q15_s2	5	27.4	-0.001	-0.088	+0.073	+0.135	+0.131	+0.050	+0.133

5.1 T1 — Capability

On the E0 gate (15 fixed Medium procedural tasks, $k=5$, 75 trials/model, 95% Wilson [24] CIs), capability separates into *two walls*: correctness ranks the low/mid band, while *speed* (**fast_rate**, the fraction beating the `min(eager, torch.compile)` roofline by $\geq 1.10\times$) is the frontier discriminator.

Model	correct	fast	mean C
GPT-5.6 Terra (api)	0.920	0.667	0.793
GPT-5.6 Sol (api)	0.907	0.600	0.753
Kimi K2.5	0.387	0.240	0.313
Fable 5.1	0.360	0.227	0.293
Palmyra-X5	0.533	0.040	0.287
gpt-oss-120b	0.467	0.080	0.273
Qwen2.5-Coder-7B (open)	0.280	0.000	0.140

Table 9: T1 capability (top rows). *Every* open model $\leq 14\text{B}$ has **fast_rate** = 0: they can sometimes write a correct kernel but essentially never a frontier-fast one; Coder-14B and Gemma-3-27B additionally hallucinate `torch` internals. Source: `leaderboard.json`.

Intuition. *The two walls matter for everything above: T1 shows that below the frontier, models are correctness-limited long before they are speed-limited. So the RSI rungs are mostly testing whether a model can compound its way up the correctness wall, not whether it can shave microseconds — which is why “where C moves, it moves via correctness acquisition, not speed” recurs throughout.*

5.2 T2 — Weight-RSI: a rigorously-powered compounding null

The title claim is where the strongest evidence lives, and it is a *null*. Over multi-round rejection-sampling SFT, carrying the lineage forward does not beat a matched reset.

Finding. The independent replicate is the *trajectory* (one seed, one base checkpoint, one held split, one accumulated adapter), not the round: rounds within a run are serially dependent, and an earlier draft of this report pooled them as if independent. Pooled across scales (0.5B–14B) over **57 trajectories** comprising 228 round-comparisons, $L-R = -0.010 [-0.035, +0.016]$, TOST-equivalent at $\delta=0.05$ with $\text{BF}_{01}=5.6$ — *moderate* evidence for the null. A cluster-robust estimate clustered on trajectory agrees ($-0.002 [-0.024, +0.020]$, $\text{BF}_{01}=7.4$). The naive round-level pooling would have reported $\text{BF}_{01}=14.7$; that figure is an artifact of the clustering and we do not claim it. The verdict is robust: restricting to trajectories with ≥ 3 rounds ($n=36$) gives $\text{BF}_{01} \approx 5.9$ and to full-length ≥ 6 -round trajectories ($n=29$) gives $\text{BF}_{01} \approx 4.9$, both EQUIV.

Scale	traj. n	rounds	$L-R$ (95% CI)	TOST	BF ₀₁
Qwen2.5-Coder-0.5B	12	51	-0.014 [-0.058, +0.030]	EQUIV	2.6
Qwen2.5-Coder-1.5B	20	93	-0.001 [-0.037, +0.035]	EQUIV	4.5
Qwen2.5-Coder-3B	19	78	-0.027 [-0.086, +0.032]	not equiv.	2.7
Qwen2.5-Coder-7B	5	5	+0.024 [-0.126, +0.174]	underpowered	2.0
Qwen2.5-Coder-14B	1	1	—	n/a (single run)	—
Pooled	57	228	-0.010 [-0.035, +0.016]	EQUIV	5.6

Table 10: Lineage vs. matched reset at the **trajectory** level (primary estimand; `equivalence_tost.py`). n counts independent trajectories; “rounds” counts the round-comparisons they contain. Intra-trajectory ICC=0.093 (design effect 1.28), so the naive $n=228$ rounds carry the evidence of ≈ 178 independent observations. Individual scales are now only moderately powered — 3B is *not* individually equivalent once clustering is respected, and 7B ($n=5$) is not interpreted. Per-round, $L-R$ shows a *transient* early advantage (rounds 1–2) decaying to ≈ 0 by round 3.

The uncontrolled T2 boards (`rsi_leaderboard.json`, `tier_speed_rsi.json`) do show apparent per-model “compounding” — the strongest being **stable-code-3b**, self-fresh-frozen +0.226, sign-consistent across 2 seeds; and starcoder2-15b +0.074 across 3 seeds. But the decisive recursion-interruption fork (A1, `fork.json`) dissolves it: for deepseek-1.3b, continuing to self-train vs. freezing the producer at the fork gives self-ckpt = -0.001 — a **one-time upgrade, not compounding**. Where C does move it is almost entirely *correctness acquisition*, not speed (`decomp.json`: deepseek-1.3b $\Delta_{\text{correct}} + 0.89$ vs $\Delta_{\text{speed}} + 0.38$).

Intuition. *The lineage-vs-reset null is the cleanest single fact in the project, and it is stronger than “updates are small.” If updates were merely small, lineage and reset would both drift slowly but lineage would still inch ahead. Instead they are indistinguishable — which means the accumulated weight state carries no useful cumulative information beyond what one round of fresh self-training recovers. That is a statement about the state, not the step size.*

An H100 replication under a correctness-reliability metric

The A100 compounding null above is measured on the headroom score. On H100 that score has no range (Section 7), so we re-ran the same protocol under a pre-registered second metric — *pass-rate*, the fraction of the k sampled candidates that verify (pre-registration, Amendment 1). Eight trajectories, 1.5B and 3B, two seeds per arm:

Arm	lineage – reset	95% CI
control (published protocol)	+0.184	[+0.080, +0.287]
inject (teacher coverage on failed tasks)	+0.184	[+0.089, +0.278]
inject – control	+0.000	—

Both CIs exclude zero: on this metric, accumulation beats a matched reset. This is the first non-degenerate compounding measurement we obtained on H100, and we report it with three limits that between them prevent it from supporting a compounding claim.

First, the search baseline is degenerate under this metric and we do not report it. Best-of- N ’s mechanism is the *maximum* over $k(r+1)$ draws; pass-rate is a *mean*, which is invariant to the number of draws. Measured: as the budget grew five-fold, $C_{\text{best-of-}N}$ moved -0.024. The apparent lineage – best-of- N = +0.676 (positive in 37/37 rounds) would therefore *reverse* our

search-beats-training result using an artifact. **A baseline is only a baseline with respect to a scoring rule**; changing the rule silently changes what every comparator means, and each must be re-derived rather than carried over.

Second, the metric saturates at the top. $C_{\text{lineage}} = 1.000$ in 18 of 37 rounds (49%) on a five-task held set. Once lineage pins at the ceiling the difference measures only how far *reset* fell below it, so the magnitude is a lower bound and the across-round trend is unreadable. Having diagnosed a floor effect and built a metric with range at the floor, we landed in a ceiling effect: **saturation is a property of the match between task difficulty and model ability, not of the metric**, and changing the metric only moves where the wall sits.

Third, the contrast is not training-compute matched. Lineage retains its adapter and trains every round ($R \times$ steps); reset re-initialises and trains on one round’s data ($1 \times$ steps). Under a metric scoring reliability of correctness, “more supervised fine-tuning on verified-correct outputs raises the rate of correct outputs” is close to tautological. The result is evidence that accumulation improves correctness *reliability*; it is not evidence of recursive self-improvement.

Finally, the positive control did not fail — it ran out of work. Injected-task counts per round run [15, 2, 1, 0, 0] and [19, 3, 0, 0, 0]: injection targets tasks the student failed, and by round 2 it solves everything, which is why the inject-control delta is exactly zero. A five-task held set that a 1.5B model exhausts in two rounds cannot support a compounding claim under any scoring rule. The remedy is the 456-task generated bank (Table 14), not a third metric.

5.3 T3 — Procedure-RSI

When a frozen-weight model rewrites its own executable procedure, frontier models improve it substantially (`trackc.json`): GPT-6 Astra Q $0.298 \rightarrow 0.854$ (**+0.556**, growing over 6 rounds), Claude Sonnet 5 **+0.401**, Mistral Large 3 **+0.310**; a frozen-procedure control (Fable-5.1) sits at +0.009. But the self-modify mechanism (`selfplay_mech.json`) shows the gain is **68% realized at round 0** (one-shot), **75% non-recursive**, with the archive saturating by round 1–2 and strategy sets already diverse (Jaccard 0.12).

Finding. The only models with *sustained* per-round procedure gain (Astra, Mistral) are the ones that started from *low* Q_0 — i.e. they are consuming headroom, not demonstrating recursive self-insight. DeepSeek-V3.2’s -0.234 “self-degrade” is a 1-round interrupted-run artifact and should not be read as a finding.

The one-shot reading is confounded by the harness, and we retract it. The procedure is a list of strategy strings, and the harness asks the model for “ ≤ 12 ” of them while displaying only 12 back. It is a *soft* request — the stored list is never truncated — so what it actually measures is instruction compliance. Frontier models comply exactly:

Model	Strategies per round	Total ΔQ
GPT-6 Astra	12, 12, 12, 12, 12, 12	+0.556
Claude Sonnet 5	12, 12, 12, 12, 12, 12	+0.401
Mistral Large 3	12, 12, 12, 12, 12, 12	+0.310
GPT-5.6 Sol	12, 12, 11, 12, 11, 11	—
GPT-5.6 Terra	10, 12, 12, 12, 12, 11	—

Five of five interpretable runs sit at 11–12 from round 0; four of five never grow at all. A procedure pinned at its limit in round 0 *cannot be observed to keep improving*, so “the model stopped improving

its procedure” and “the harness stopped letting it” are not separable in this design, and the one-shot interpretation is not earned by these data.

Re-running the same harness with an open model at caps {12, 48, unbounded} shows procedures *do* grow when permitted — 6 of 6 clean runs grow, two exceeding 12 strategies — confirming that the cap binds something real. It does **not**, however, produce a dose-response in quality: on the clean runs mean ΔQ is -0.046 at cap 12, $+0.021$ at cap 48 and -0.009 unbounded ($n=2$ per cell, all six runs complete at six rounds). Lifting the cap lets the procedure grow without making it better.

We flag a correction here rather than silently reporting the clean figures. An earlier version of this analysis reported a monotone rise to **+0.482** unbounded. That came from runs whose data directory sat on an inode-exhausted filesystem, where the reference build raised `EDQUOT` and the grader returned `False` for every candidate: those runs have $Q_0 = 0.000$ and four to five exactly-zero rounds, so their apparent “gain” was recovery from a spurious zero baseline. We had already documented that failure mode (it is defect 5 in Table 13) and used the affected runs regardless. The exclusion is now enforced in the analysis script rather than left to memory: any run with $Q_0 = 0$ or two or more exactly-zero rounds is dropped, loudly.

We state the limits plainly. There are two seeds per cap cell; open-versus-closed is confounded with model family; and because the cap is a soft request, the open model does not comply with it reliably, so these runs corroborate rather than replicate. The strong evidence is the closed-model pinning, which is visible in the *already-published* run records and requires no new experiment. Settling the question properly requires re-running frontier models at several caps. Until then we report the procedure gains as real and withdraw the claim that they are one-shot.

5.4 T4 — Closed→Open

A closed researcher rewriting an open trainee’s training harness can help substantially (`combined_rsi.json`): Fable-5.1 → Qwen-1.5B reaches improved–frozen **+0.472** (peak 0.472, C 0.147 → 0.501 vs frozen 0.029), and Fable-5.1 → Qwen-7B **+0.313**. But the sign is *pair-dependent*: the same researcher *hurts* other trainees (Fable→OpenCoder-1.5B -0.265 , Fable→DeepSeek-1.3B -0.444), and many researcher rows are flat. As with T4’s cousin T2, an open question (ROADMAP A4) is whether the $+0.472$ is correctness rather than speed.

Intuition. *Improvement does transmit through authored tooling — a closed model that cannot change its own weights can still cause a better open model. But it is brittle and trainee-specific, which reads less like “the researcher discovered a better training recipe” and more like “it happened to match this trainee’s correctness headroom.” Transmitted improvement is real but not yet a reliable, general channel.*

5.5 T5 — Self-play: an author-yield failure, not a measured null

The primary signal is $L-F$ (extra gain from a co-evolving author beyond an escalating curriculum). $L-F$ is only defined if the live author actually produced accepted, valid, novel tasks. **It did not, at the yield needed to measure anything.**

Open (`selfplay.json`, 24 runs): most models $\leq 15B$ report $L-F \approx 0$. Requiring at least five accepted model-authored tasks before a row is interpreted, only 6/24 open runs qualify. **Closed** (`selfplay_closed.json`, procedure channel, 11 runs): 10/11 rows are *exactly* $L-F = 0.0$ — including runs whose authors proposed 36–37 tasks across 24 rounds. An exact zero across 24 rounds with non-zero proposal counts indicates the co-evolution channel never engaged, not that it

engaged and returned nothing. **Open-ended** (`open_rsi.json`, 16 runs): all verdicts are “one-time (open = fixed)”.

Finding. The two rows previously quoted as positives do not survive the yield floor: starcoder2-15b $L-F = +0.232$ rests on **one** accepted authored task over 6 rounds, and deepseek-coder-6.7b-base $L-F = +0.154$ on **three** over 3 rounds. Their per-round $L-F$ traces oscillate in sign. We therefore withdraw both from the headline claims and report T5 as *undefined at the current author yield*.

What the dedicated diagnosis run actually shows. The self-play diagnosis (`selfplay_diag.json`, Qwen2.5-Coder-3B, 8 rounds) authored **zero** valid tasks in every round. Consequently every per-round diagnostic in that file — authored solve-rate, trivial and unsolvable fractions, authored headroom — is `null`, and all three arms scored $C=0$. An earlier draft of this report read that run as showing a *bimodal* authored-difficulty distribution (tasks either trivial or unsolvable, rarely in a learnable band). **That conclusion is not in the data and we retract it:** the bimodality is a plausible hypothesis the run was unable to test, because nothing was authored to measure. The supported statement is narrower: *free-form task proposal collapses*. Without constrained, executable task mutation, the author does not manufacture a frontier, so self-play cannot be tested at all — which makes structured mutation with validity and learnability gates a prerequisite experiment, not an extension.

(Note: an earlier README listed closed Sonnet-5 $+0.045$ / GPT-5.6-Sol $+0.041$ as “emerging positives”; these are *not* in the current data and have been removed.)

6 Cross-cutting analyses: the mechanism

The rung-by-rung results tell us *whether* improvement compounds. The following analyses tell us *why* it does not, and together they form the mechanistic core of the report.

6.1 Coverage vs. scale, and the “correctness wall” as a sampling artifact

Per-task p -maps on a fixed 29-task bank (`pmap_curve.json`) trace coverage (tasks with ≥ 1 verified-correct sample) from 13.8% at 0.5B to 86.2% at 14B, and show that below the wall $\text{pass}@K$ dwarfs $\text{pass}@1$:

Size	K	coverage	$\text{pass}@1$	$\text{pass}@K$	K -ratio
0.5B	64	13.8%	0.009	0.109	$\sim 12\times$
1.5B	64	27.6%	0.014	0.235	$\sim 17\times$
3B	64	82.8%	0.112	0.777	$\sim 7\times$
7B	48	82.8%	0.079	0.799	$\sim 10\times$
14B	24	86.2%	0.655	0.862	$\sim 1\times$

Table 11: Coverage and sharpening across scale. The sharpening geometry (`results_auto.tex`): the *sharpenable* band ($0 < p < 0.2$) is 14%/28%/59%/79%/0% at 0.5/1.5/3/7/14B — it peaks mid-scale and collapses at 14B, where all mass is already *reliable* ($p \geq 0.2$).

Intuition. *The sub-2B “correctness wall” is not an inability — it is a low per-sample success probability. Give a small model enough draws and correct kernels appear ($\text{pass}@K \gg \text{pass}@1$). This reframes the entire problem: self-training does not need to learn to be correct, it needs to reliably reproduce a correctness that is already latent in the sampling tail. That is a sharpening problem, not an exploration problem.*

6.2 Zero-score forensics: the wall is a formation failure

Classifying *why* each 0-score candidate failed (`forensics_summary.json`) shows the sub-3B wall is a **kernel-formation** wall (no parseable ModelNew), not runnable-but-wrong code: `no_extract` dominates at 0.5B (72%), DeepSeek-1.3B (90%), Yi-1.5B (96%). The failure *locus marches downstream with scale*: incoherence $\rightarrow$ truncation/syntax (Qwen-7B: 75% syntax) $\rightarrow$ API-hallucination $\rightarrow$ wrong-output $\rightarrow$ correct (Qwen-14B: 65% correct, only 0.2% `no_extract`).

Intuition. *Small models do not fail by writing subtly-wrong kernels — they fail by not producing a well-formed kernel at all. That is why coverage is so low and why self-training starves: there is nothing correct to train on. As scale rises the bottleneck migrates from “can it write a kernel” to “is the kernel right” to “is it fast.”*

6.3 Search beats training at matched compute

Analysed at the trajectory level over 57 trajectories / 228 round-comparisons (`search_vs_train.json`), lineage—bestof $N = -0.111 [-0.138, -0.083]$ (CI excludes zero; the cluster-robust estimator, which weights rounds rather than runs, gives $-0.142 [-0.163, -0.120]$ — a larger effect, same sign, and its interval also excludes zero), and verified search beats lineage in **84% of runs** (76% of rounds); every well-powered scale is individually negative (0.5B -0.123 , 1.5B -0.114 , 3B -0.123 ; 7B -0.018 at $n=5$ is not interpreted). Unlike the compounding null, this positive *strengthens* under the clustering correction, because the contrast is made within a round. A separate matched-budget baseline sweep (`baselines.json`, budget 20) is weaker and mixed — Qwen-1.5B recursion_gain $+0.055$ over its best non-recursive baseline, but starcoder2-3b -0.073 — so the strong, consistent statement is the search-vs-lineage one.

Finding. At a matched compute ledger, best-of- N verified search beats lineage self-training (lineage—bestof $N = -0.111 [-0.138, -0.083]$ over 57 trajectories, CI excludes zero). The useful solutions are *already reachable by sampling*; search exploits them directly, while training must compress them into shared weights and have the gain survive.

6.4 WHY-RSI: the weight-level mechanism and the inverted-U in scale

Across 133 weight-RSI runs (`mech_analysis.json`) the correctness wall is the primary gate: crossing rises from 54% below 2B to 92% at ≥ 2 B. Among the 97 wall-crossers, RSI is an **inverted-U in scale**:

Band	n	wall-cross	LoRA drift	retention	frac RSI
<2B	67	0.54	0.019	0.013	0.03
2–8B	40	1.00	0.271	0.180	0.50
≥ 8 B	26	0.81	0.596	0.339	0.23

Table 12: The inverted-U (`mech_analysis.json`, $n=133$). RSI peaks at 2–8B; the largest models drift *most* in weight space yet compound *least* — motion without progress near the roofline. 28/133 runs met the compounding criterion (stable-code-3b recurs). When drift occurs it localizes to *late* layers (task adaptation) and the held-family transfer gap is ≈ 0 (no transferable meta-skill).

Finding. A subtle and important nuance: on this bank, compounders have *lower* generation diversity than flat runs (0.41 vs 0.52), not higher. The discriminator between compounding and

flat among wall-crossers is **sustained LoRA drift + retention** ($\sim 5\times$ higher in compounders), *not* diversity preservation. So “diversity collapse causes the null” is *not* supported by our weight-RSI data — compounding here looks like *productive convergence onto a good basin*. Diversity contraction remains a plausible *cross-round option-value* lock (see the Discussion) but we flag it as an association whose clean test is a diversity-preserving intervention, and we explicitly do *not* claim it as the mechanism.

Intuition. *Scale does not monotonically help RSI — it is an inverted-U. Small models cannot form correct kernels often enough to feed the training set (formation-starved). Mid-scale models sit on the widest learnable frontier and compound best. Large models cover almost everything already, so their weights churn (high LoRA drift) without moving the held-out score — “motion without progress” near the roofline. The binding constraint slides from formation (small) to headroom (large).*

6.5 Probe-as-intervention: correctness is represented better than it is generated

A linear probe reads “is this kernel correct” out of mid-to-late layers at high AUC (Qwen-3B best_auc **0.985** at depth 0.86, while its actual correctness rate is 0.105) — the model *represents* correctness far better than it *generates* it (`interp.json`).

Finding. Analysed at the right unit, probe-guided selection *does* beat uniform-random selection at matched *generation* budget. Clustering the 19 usable runs on their 12 distinct base checkpoints (rows sharing a checkpoint are repeated draws, not distinct models), the mean lift is **+0.124** [**+0.029**, **+0.218**] — the 95% CI excludes zero — recovering 36% [11%, 60%] of the oracle–random headroom. An earlier draft read the same rows as “no advantage” because it looked at the spread of individual runs (−0.27 to +0.45) rather than the clustered mean. See Appendix B.

Three reasons this is a modest secondary result, not a headline. (i) *Uniform random is the weakest possible comparator*: the informative ones — length-normalised log-likelihood reranking, a compile/run validity filter, execution agreement — were not run, so we cannot say the probe is the best use of the same compute. (ii) *The denominators are tiny*: each run’s lift is computed over 5–12 test tasks, so a “+0.45” is about five problems, and the clustered interval is correspondingly wide. (iii) *Matched-generation selection is not matched-verification harvesting*: a gain in top-1 correct rate from reranking an existing pool does not imply the probe makes a verification-constrained search cheaper.

What we cannot verify from the released artifacts. Two figures quoted in earlier drafts — a within-task ranking AUC of ≈ 0.67 and a matched-verification harvesting yield of +0.01 — come from a probe-baselines run whose per-candidate outputs are *not* in this repository. Per-candidate probe scores were not retained anywhere, so the within-task AUC cannot be recomputed from what we release. We therefore do not rely on either number, and we state the within-task estimand as **unmeasured** rather than as measured-and-small. This is the single most important gap in the probe story: only a within-task ranking metric can explain a selection gain mechanistically, and the pooled AUC cannot substitute for it — as the runs with pooled AUC=1.000 and lifts anywhere from +0.000 to +0.250 demonstrate.

Intuition. *A linear probe can read “is this kernel correct” out of mid-to-late layers at high AUC, yet the model emits a correct kernel far less often than it can internally recognize one. The model knows more than it generates. This is the same discovery→assimilation gap seen from the inside: the limiting step is turning latent competence into a reliably-produced output.*

Defect	Presented as	Caught by
Grader mis-imported under a wrong root	“self-training does not compound”	identity-kernel assertion
Grading device index absent on a 1–2 GPU allocation	“the model writes bad kernels”	same assertion, run <i>on the allocation shape the jobs use</i>
Raw generations graded without extracting the submission	“the 14B teacher solves 0/29”	non-zero coverage precondition
Score saturated at the correctness floor	“lineage $\approx$ reset”	score-histogram check
Compiled-baseline cache hitting a filesystem inode quota	“4 of 6 rounds produced nothing”	output-path/quota correlation
Our own explanation of the first five	“H100 closed the headroom”	tracing prompt $\rightarrow$ number
Host CC leaking into the container, so Triton [20] could not build	“models cannot write kernels when asked”	known-good <i>Triton</i> kernel

Table 13: Seven defects, each of which independently manufactured a null. The middle column is what the artifact would have claimed had the defect not been found. Every one of these is a statement we came close to publishing.

7 Instrument validity: how a self-improvement benchmark manufactures a null

A benchmark that reports a null owes its reader a second result: evidence that the measurement *could have* reported a positive. Self-improvement benchmarks are unusually exposed here, because the headline quantity is a difference between two arms of the same pipeline. Any defect that suppresses both arms equally leaves the contrast at zero while every surface diagnostic — jobs complete, kernels compile, scores populate — looks healthy. A broken instrument does not report that it is broken; it reports $\Delta \approx 0$.

Over this project, **seven independent pipeline defects each produced a plausible, publishable-looking null**, summarised in Table 13. None announced itself, and three survived a code review before being caught by an assertion. Six were found only after they had been written up as findings; one was caught before it reached prose, and we report what made the difference.

A clean zero is a bug hypothesis before it is a finding. The operational signature that caught most of these is distributional, not statistical. A *real* null is noisy: it has per-task spread, partial successes, seed-to-seed variance. A null in which every task fails identically, or every score takes the same value, is instrumentation. We therefore treat suspicious tidiness as evidence *against* the result, inverting the usual intuition that clean data is good data.

Correctness of the instrument is not sufficient; it must also have range. This is the failure mode we consider most transferable, because it defeats the obvious check. Our headroom-normalised score is $0.5 + 0.5 \cdot \text{clamp}\left(\frac{sp-1}{\text{ceiling}-1}\right)$, which is well-formed, was verified end-to-end on known-good inputs, and was working exactly as specified. It was nonetheless useless as a contrast: on H100, **75% of the frozen base’s *non-zero* scores on the 29-task bank land within ± 0.01 of parity** (18 of 24 solved tasks) — precisely the value of a correct submission at parity with the baseline. With both arms pinned to the same absorbing value, lineage — reset is zero *by*

construction, and the experiment could not have detected compounding in either direction.

The sixth “defect” was our own explanation — and so was its correction. This entry has been wrong twice, in opposite directions, and we report the sequence because the second error is more instructive than the first.

First claim. We attributed the 0.50 spike to `torch.compile` being relatively stronger on H100, closing headroom that existed on A100.

First correction, which we published. We then found that the scorer takes the compiled baseline only under `KA_SCORE=compiled`, and that its `ceiling` argument defaults to 1.5 when unsupplied — and concluded that the runs had used the eager baseline with a fixed anchor, so that “no claim about `torch.compile` was licensed by those runs”.

That correction was false. The cluster tooling exports `KA_SCORE=compiled` into *every* job through its project environment, and has since before any of these runs. The submitted batch scripts contain the line. Those runs therefore *did* score against the `torch.compile` baseline with the per-task roofline ceiling — exactly as pre-registered. We had checked the library default and never checked the environment the jobs actually ran in.

What the evidence now constrains. A score of exactly 0.50 requires $sp_{\text{compiled}} = 1.0$ *exactly* — the submission matching the compiled reference. That is consistent with two very different worlds: one where `torch.compile` is strong and the model matched it, and one where `torch.compile` gives no speedup on these memory-bound elementwise operators, so the compiled baseline *is* the eager baseline and a plain-torch rewrite lands at $1.0\times$ by construction. These are distinguishable without any model: measure $t_{\text{eager}}/t_{\text{compiled}}$ per task. That probe is running, and we report its result rather than choosing between the two stories now.

The transferable lesson is about where a configuration lives. We reasoned from the library’s default argument and the code path, which is where a programmer looks, and the answer was in the job’s environment, which is where the program actually ran. **A default is not a setting.** An experiment’s configuration is the union of its code, its invocation and its environment, and only the last of those is invisible from inside the repository. This is the same failure as the seventh defect below, where the host’s `CC` leaked into a container — both are the environment silently overriding what the code appears to say.

Settling it by measurement: the compiled baseline is a real baseline. Rather than choose between the two stories above, we measured the quantity that separates them — $t_{\text{eager}}/t_{\text{compiled}}$ per task, with no model involved:

<code>torch.compile</code> gain over eager	tasks (of 29)
$< 1.1\times$ (no meaningful gain)	0
$1.1\text{--}1.5\times$	15
$> 3\times$	14
median $1.34\times$, max $21.85\times$	

`torch.compile` beats eager on *every* task in the bank. So the compiled baseline is not a naive reference, and the second story is the right one: a submission scoring 0.50 under the compiled scorer has **matched an optimised baseline**, which is a meaningfully stronger statement than we had been making. The instrument was measuring something harder than we credited it with.

This also resolves the numbers that looked contradictory. Under the *calibration* scorer (eager, fixed $1.5\times$ anchor, which `difficulty_filter` invokes directly and which no environment variable

affects), a plain-torch rewrite lands at exactly parity — hence the cluster at 0.50. Under the *run* scorer (compiled, per-task roofline), the same rewrite scores *below* parity, which is why the frozen-base capability on those runs is 0.401 and not 0.50. Two scorers, two distributions, one of which we had been quoting as the other.

The seventh defect: the benchmark had never measured its own subject. Having traced the plateau to the prompt, we tested the obvious follow-up — remove the steer and ask explicitly for a Triton kernel. The model solved **0 of 29** tasks, against 25 of 29 under the published prompt. That is a clean, pre-registered, publishable result: the models try when asked and cannot produce a working kernel.

It is also false. Before reporting it we applied the known-good-input gate to the *new* input: we hand-wrote a Triton kernel that is correct by construction and required the real grader to verify it, alongside a plain-torch control that must pass and a wrong kernel that must fail. The controls behaved; the correct Triton kernel returned `FileNotFoundError`. The cause was that Triton compiles a small C shim at first use and takes its compiler from `$CC`, which the cluster’s module system sets to a *host* toolchain path that does not exist inside our container. Plain PyTorch never compiles anything at runtime, so **only Triton broke** — and Triton is the subject of the benchmark. One environment variable fixed it, after which the same kernel verifies.

The consequence is not a corrected number but a corrected scope: for the entire history of this project, **every submission that was not plain PyTorch failed for an environment reason**. The benchmark has been measuring correctness-preserving rewriting, not kernel optimisation.

This also revises our own account of the previous defect. We attributed the saturated score to the prompt’s instruction to prefer plain torch — but that instruction now reads as a *rational adaptation to a broken grader*. Tuned against a harness in which Triton always failed, plain torch genuinely was the only thing that worked. The prompt and the grader were coupled, and the grader came first. Generalising: **a configuration that looks inexplicably conservative may be compensating for a defect**, and because the defect never appears in any result, the workaround looks like a design choice indefinitely.

Two things made this one catchable before publication rather than after. First, failure reasons now propagate: the grader had always computed a reason string and discarded it one line later, so every failure in project history was a bare `False` — and “model writes Triton, Triton grades `False`” reads naturally as a statement about the model. Second, and more transferable: **when an experiment begins producing a new *kind* of artifact, the known-good gate must be re-instantiated for that kind**. “The grader works” was true and useless; “the grader works on Triton” was the question, and nothing was asking it.

What the corrected instrument then showed. With Triton verifying, the prompt A/B could finally be run as intended. Classifying every generation (rather than counting only successes) across four tasks at $k=4$, $n=16$ per arm:

Prompt	Outcome	$n/16$	95% CI
published	plain-torch rewrite	16	[81%, 100%]
published	attempted a custom kernel	0	[0%, 19%]
kernel	attempted a custom kernel	14	[64%, 97%]
kernel	of which <i>verified correct</i>	1	—
kernel	of which failed verification	13	—

This separates two quantities the project had conflated throughout. **The prompt controls whether the model attempts a kernel at all** — 0 attempts versus 14 is an instruction being followed, not a capability difference. **Capability controls whether the attempt succeeds** — 1 of 14 verified. The model can write a working Triton kernel and usually does not. With $n=16$ the interval is wide and the point estimate is not the claim; the claim is that the rate is *neither zero nor high*.

That distinction is the most consequential measurement in this report, because it explains every saturation we chased. The 0.50 floor, the ceiling at 1.000, and a coverage-injection control that exhausts its task set by round 2 all share one cause: **these models have already saturated the task the benchmark was actually measuring**, which is correctness-preserving rewriting rather than kernel optimisation. Any metric built on a saturated task piles up against one wall or the other, and we tried three — a headroom score, a pass-rate score, and a 456-task generated bank — before recognising that.

The substrate problem was therefore never a metric problem. The remedy is not a better scoring rule but a harder task, and the harder task only became measurable once the grader could verify one. We regard this as the central practical lesson of the work: when a population sits against a wall under every metric you try, the task is too easy, and no amount of re-normalising will manufacture the variance that is missing.

The loss cascade: where submissions actually die. A benchmark that reports “the model solved n of N tasks” compresses a pipeline with several independent failure stages into one number. Decomposing it changed what we believed at three separate points, so we report the decomposition as a required artifact.

For a 0.5B model under the kernel prompt, $k=12$ over 29 tasks:

Stage	Survivors	Loss	Dominant cause
generations	348	—	
parse into a submission	118	66%	truncation (39% of all generations)
<i>after raising the budget</i>	—	—	wrong class name (67%)
attempt a kernel	74	37%	the model complies
verify	4	95%	incorrect kernels

Three facts fall out that the solve-rate number cannot express.

The largest loss was a configuration constant. Generation was capped at 900 tokens, which is comfortable for a plain-torch rewrite and insufficient for a Triton kernel; 39% of generations were truncated mid-output and discarded. Raising it to 2048 drove truncation to 5%. A fixed budget is a confound whenever the artifact being measured has a length, and it biases *across scale* because larger models write longer code.

Fixing the top cause did not fix the loss. Extraction rose only from 38% to 41%: the generations that previously truncated now complete, and name their class `Model` — the reference’s own name — rather than `ModelNew`. Truncation had been masking a larger failure. We report this because the natural write-up (“raising the budget recovered the pipeline’s largest loss”) would have been true of the mechanism and false of the outcome.

The stages fail differently at different scales. At 14B, extraction is 95% — there is no truncation problem — and *zero* of 331 parsed candidates verify. The small model dies before the grader; the large model dies at it. A single solve-rate column reports both as “low capability” and reports the 0.5B as the *better* model, which is how we came to publish an inverted capability curve.

Reachability: the dynamic-range precondition, made quantitative. “Check the metric has range” is advice until it is a number, so we propose one. **Reachability** is the fraction of bank tasks on which the *frozen base* already produces a submission clearing the scoring anchor. It is cheap — it falls out of the same calibration pass used to admit tasks — and it is computed before any experiment runs.

Measured on the two banks we calibrated against an identical H100 anchor:

Bank	Tasks	Admitted	Reachability	
Hand-curated (29-task)	29	23	1 / 29	(3.4%)
Generated DSL (456-task)	456	180	57 / 456	(12.5%)

Table 14: Reachability = fraction of tasks where the frozen base clears the scoring anchor ($1.5\times$ over eager). Both banks are bimodal — parity, or well past the anchor — but they differ four-fold in whether the speed dimension is reachable at all.

A reachability of 3.4% means that on 28 of 29 tasks, *every* success the base produced landed at parity, so the score had one attainable non-zero value and the contrast between two arms was pinned before a single round ran. We would now decline to run a speed-scored compounding study on such a bank, and we report the statistic for any bank we publish.

Note what this does *not* say. The measurement above is against eager PyTorch at a fixed $1.5\times$ anchor, which is what the default scorer uses; the roofline ceiling does not enter it. It therefore licenses no claim about `torch.compile`, and a bank of *generated* references may be easy to beat for reasons that do not transfer — a naive reference is not a strong baseline. Reachability is a necessary condition for a usable substrate, not a sufficient one.

Applying the precondition to our own headline. It would be evasive to propose reachability and not report it for the bank behind our own A100 null. Per-task frozen-base scores were not retained for those runs — an artifact-retention failure we record as such — so task-level reachability cannot be reconstructed. What was retained is C_0 , the frozen-base mean on each run’s held split:

Scale	Runs	Median C_0	Runs with $C_0 > 0.51$
0.5B	12	0.100	0%
1.5B	20	0.100	0%
3B	19	0.381	0%
7B	5	0.700	60%
14B	1	0.599	100%
All	57	0.200	7%

This matters because it shows the A100 substrate does *not* carry the H100 failure signature. There, scores piled up at exactly 0.50 — correct-at-parity, an absorbing state. Here the median is 0.20, well *below* parity, rising monotonically with scale. Most held tasks were not solved at all rather than solved-at-parity, so the measurement had room to move in the direction that mattered and the saturation critique does not transfer.

We state the cost of that defence in the same breath. A frozen base at $C_0 = 0.20$, with only 4 of 57 runs above parity, means the *speed* dimension was barely exercised on A100 either: most of the available signal was correctness, not optimisation. The A100 null is therefore best read as a null about *acquiring correctness through self-training*, which is a narrower claim than the one the original framing implied — and narrower than the title promised.

The general point is uncomfortable and worth stating plainly: **we could apply our own proposed precondition to our own headline only partially, and only because a coarser statistic happened to survive.** A precondition is worth little if the artifacts needed to evaluate it are not retained, so we now record per-task frozen-base scores and stamp every artifact with the environment that produced it.

What we propose as standard practice. We ship these as executable gates rather than advice, and we recommend them for any self-improvement benchmark reporting a null:

1. **Known-good input, known-good output**, asserted before every batch and run *on the same allocation shape as the experiment* — one of our defects lived entirely in the difference between a 1-GPU and a 3-GPU allocation.
2. **A dynamic-range precondition**, reported as *reachability* (Table 14): the fraction of tasks on which the frozen base already clears the scoring anchor. Publish it beside every bank, and never interpret a contrast computed on a bank where it is near zero.
3. **A positive control on the causal path**: an intervention known to move the primary metric (for us, injecting verified solutions on tasks the student failed). A null is interpretable only once the harness has been shown to register a positive.
4. **Mechanism denominators reported alongside every rate**, so undefined is distinguishable from zero.
5. **Automated cross-artifact consistency checking**: every number in the manuscript re-derived from the data files at build time. Ours catches stale figures and claim strings that contradict the data they cite — both of which occurred.

We emphasise that this is not a catalogue of unusual carelessness. Each defect was a single line, each was locally reasonable, and each was caught only because an assertion was looking for it. The generalisable claim is structural: **in a benchmark whose headline is a difference between two arms of one pipeline, the cheapest result to produce accidentally is a null**, and the field currently has no convention requiring authors to rule that out.

8 Synthesis: why self-improvement does not compound here

8.1 The discovery-to-assimilation bottleneck

The mechanism: a discovery→assimilation bottleneck (sharpening, not exploration).

Our two strongest results jointly localise the failure. Best-of- N search beating lineage training at matched compute (lineage–bestof $N = -0.111 [-0.138, -0.083]$ over 57 independent trajectories), together with the large $\text{pass}@K \gg \text{pass}@1$ gap, shows that *useful solutions are already present in the sampling tail* but are not reliably produced in one sample. The limiting pipeline is therefore

rare verified sample $\rightarrow$ selected SFT example $\rightarrow$ weight update $\rightarrow$ reliably reusable gain,

and it is *lossy*: search can exploit a rare discovery directly, whereas training must compress it into a shared parameterisation, preserve other skills, and have the gain survive later rounds. The lineage-vs-reset null (inheriting updates does not beat restarting) is direct evidence *against a useful cumulative learning state*, not merely evidence that individual updates are small. This is the classic self-training signature: selected-sample training concentrates probability on *already-reachable* successes faster than it expands what is reachable (cf. STaR [25], ReST, ReST^{EM}, expert iteration; and the RLVR capability-boundary debate, Yue et al. 2025).

A formal fixed point — with its limits stated precisely. Under an *idealised* support-preserving rejection-conditioning operator R (unlimited accepted samples, exact per-task fitting, no cross-task transfer), exact solution-support coverage $C_0(\pi) = \Pr_x[p_\pi(x) > 0]$ is invariant and R is idempotent after one step: $C_0(R\pi) = C_0(\pi)$ and $R^2\pi = R\pi$. On tasks with positive success probability R places all mass on accepted outputs; on $p=0$ tasks no accepted distribution exists. So an idealised verified-SFT loop is a *one-step fixed point in exact coverage*. **We are careful about what this does *not* say.** (i) Neural SFT is not this operator — shared parameters can generalise and raise the probability of unsampled outputs, so real SFT need not preserve support. (ii) *Finite-budget* coverage is *not* invariant: $C_K(\pi) = \mathbb{E}_x[1 - (1 - p_\pi(x))^K]$ rises sharply under sharpening even when C_0 is fixed — “pass@ K coverage is a fixed point” is false. (iii) Zero observed successes only bound $p \lesssim 3/K$; our “unreachable mass” is more precisely *undiscovered mass at the evaluated budget*. (iv) A training-operator fixed point is *not* a parameter-space local optimum. We therefore report an **operational self-training plateau / protocol-relative fixed point**, never an unqualified “local optimum.”

Why verified search escapes where training does not. Search does not escape the policy’s support; it escapes the *one-sample* bottleneck and avoids the assimilation bottleneck. For success probability p , $P(\text{hit in } N) = 1 - (1 - p)^N$ makes extra draws valuable when p is small, and the verifier keeps the winner. Search (a) uses a discovery directly rather than compressing it into weights, (b) allocates candidates per-task rather than via one shared update, and (c) does not permanently contract the proposal policy. The precise statement: *verified search accesses useful existing tail capability more efficiently than the tested training loop consolidates it*. (This does not settle lifetime economics — a training update could amortise over enough future queries; that needs a separate deployment-horizon study.)

Diversity contraction as a reinforcing lock (associational). Selected-sample SFT can drive a loop: current common winners $\rightarrow$ accepted set $\rightarrow$ higher probability on those winners $\rightarrow$ fewer alternatives in the next accepted set. The loss is *option value* — rare decompositions and kernel families that would enable *later* discoveries. If undiscovered useful mass U_t shrinks, later rounds

find fewer novel successes ($1 - (1 - U_t)^N \leq NU_t$) even as pass@1 rises on familiar ones. Higher retention among compounders and diversity collapse at large scale are *consistent* with this lock, but we flag them as association, not proof: low diversity can also reflect successful specialisation, and the cleanest test is a matched-reward, matched-example, diversity-*preserving* intervention.

The inverted-U, mechanistically. Partition tasks into *undiscovered* ($p \approx 0$ at budget), *learnable frontier* ($0 < p < 0.2$), and *near-ceiling* (p high). Most self-training value comes from the middle band. Small models: formation failures ($P(A | x) = P(F | x)P(A | F, x)$ with tiny $P(F)$) starve the accepted set; mid-scale: the frontier is widest, so discoveries become signal (RSI peaks); large scale: high coverage but little remaining headroom, so drift redistributes mass within already-covered behaviour (“churn without gain” — supported descriptively; “only superficial late-layer edits” is *not* yet established, as parameter distance is not calibrated functional change).

What would break the plateau, and what our data predict. Two distinct breakouts are needed: *accumulation* (retain/express more already-discoverable successes — could yield lineage>reset) and *frontier expansion* (make new strategies operationally reachable — required once the frontier is exhausted). A productive loop needs new discoveries + assimilation + retention + headroom simultaneously. The single most informative next experiment is a **coverage-injection** $\times$ **diversity-preserving** factorial: inject stronger-model / repair / search-generated solutions on *undiscovered* tasks and test whether inherited gains then emerge — separating “lack of discoveries” from “loss of discoveries after learning.” Exploration bonuses, novelty-selected accepted sets, prerequisite curricula, off-policy replay, and search-amplified expert iteration are ranked candidates; each helps only the specific deficit it targets (e.g. entropy cannot fix formation; off-policy replay of redundant winners adds no frontier information).

Scope — claims we do *not* make. We do not claim the models’ reachable set is mathematically invariant, that the weights sit at a genuine local optimum, or that RL/RLVR cannot expand capability. We claim, with *moderate* evidence for the null and a significant positive respectively: under verified rejection-sampling SFT at the tested scales/budget, carrying the lineage forward does not beat a matched reset (trajectory-level TOST [17] equivalence at $\delta=0.05$, $BF_{01}=5.6$ over 57 independent trajectories); verified search is a better use of the same compute (CI excludes zero); and the binding internal signals are coverage/formation (small), a mid-scale learnable frontier, and drift-without-gain plus diversity contraction (large). The synthesis: *expert iteration without sufficient expert amplification, plus incomplete retention of the apprentice’s useful tail.*

9 The accumulated intuition (what three weeks taught us)

This section is the part that does not fit in a conference paper: the qualitative model we now carry in our heads about RSI in this domain.

1. **Self-training is a sharpening operator, not an exploration operator.** It concentrates probability on already-reachable successes faster than it expands what is reachable. Everything else follows from this.
2. **Coverage is the currency, and it is not self-generated.** A round of self-training can only reuse solutions the model already finds; it cannot manufacture coverage on tasks where the per-sample success probability is effectively zero at the evaluated budget. Compounding is coverage-limited, and coverage is exogenous to the loop.

3. **The bottleneck is discovery→assimilation, not knowledge.** The model represents correctness it cannot reliably generate (probe result), and search — which uses discoveries directly — beats training, which must assimilate them into weights and retain them. The lossy step is assimilation-and-retention.
4. **“Operational plateau,” not “local optimum.”** We are careful: the null is a protocol-relative fixed point of the tested verified-SFT loop, not a claim that the weights sit at a parameter-space local optimum. Finite-budget coverage is *not* invariant; a better loop could still move.
5. **Diversity contraction is a plausible reinforcing lock.** Selected-sample SFT can raise probability on current winners at the cost of the rare decompositions that would enable *later* discoveries. We flag this as association, not proof — the clean test is a diversity-preserving intervention.
6. **The open contribution is a better *harness*, not a better model.** Because RSI is a protocol we run, the way to break the plateau is to change the protocol: inject coverage on undiscovered tasks, preserve diversity in the accepted set, amplify the “expert” with search before assimilation. This is exactly what the public benchmark’s harness-submission track exists to surface.

10 What would break the plateau: predictions

Two distinct breakouts are needed and they target different deficits: *accumulation* (retain and express more already-discoverable successes — could make lineage finally beat reset) and *frontier expansion* (make genuinely new strategies operationally reachable — required once the current frontier is exhausted). A productive loop needs new discoveries + assimilation + retention + headroom *simultaneously*. Our single most informative next experiment was, at the time of writing, a **coverage-injection** × **diversity-preserving** factorial. The instrument work in Section 7 has since superseded that. The coverage-injection control was run, and its injected-task count falls to zero by round 2 because the student solves the entire held set; injection cannot separate “lack of discoveries” from “loss of discoveries” on a task the model has already saturated.

The informative experiment is now the one a repaired grader makes possible for the first time: run the compounding protocol on **kernel authoring** rather than correctness-preserving rewriting. Under the published prompt, models attempt a custom kernel in **0 of 16** samples; when asked explicitly, **14 of 16**, of which **1** verifies. A success rate in that band is far from both the floor and the ceiling that defeated every metric we tried, which is precisely the regime a compounding study requires. Whether self-training compounds on a task the model has *not* saturated is, as far as we can tell, still open — and it is a question this benchmark could not previously have asked.

11 Limitations and scope

We claim, with moderate evidence on the conventional thresholds [8] ($\text{BF}_{01} = 5.6$ at the trajectory level): under verified rejection-sampling SFT [25, 6] at the tested scales and budgets, carrying the lineage forward does not beat a matched reset; verified search is a better use of the same compute; and the binding internal signals are coverage/formation (small scale), a mid-scale learnable frontier, and drift-without-gain plus diversity contraction (large scale). We do *not* claim the reachable set is mathematically invariant, that the weights sit at a genuine local optimum, or that RL/RLVR

(as opposed to rejection-sampling SFT) cannot expand capability. All primary intervals are now computed at the *trajectory* level rather than the round level, since rounds within a run share a base checkpoint, a seed, a held-out split and an accumulated adapter; this is the honest unit of replication and it costs us evidential strength, moving the equivalence Bayes factor from $\text{BF}_{01} = 14.7$ (pooling 228 rounds) to 5.6 (57 trajectories) — strong to *moderate*. The verdicts survive the correction, and the search-beats-training positive strengthens under it, because that contrast is made within a round. Per-trajectory (2–3 seed) uncertainty remains the binding constraint. Finally, all results here are A100; our H100 runs are reported separately and are non-comparable on two axes at once: different roofline constants, and a harness that silently rejected every non-PyTorch submission (Section 7). An earlier version of this sentence blamed a stronger `torch.compile` baseline; that attribution was wrong — the default scorer never used the compiled baseline — and is retracted in Section 7.

12 Conclusion

Three weeks of clean-verifier, compute-matched experiments across five rungs of recursion and scales from 0.5B to 14B converge on one picture: in GPU-kernel optimization, verified self-training *sharpens what a model already covers but does not explore*, so improvement does not compound — not because the models lack capability, but because the loop’s feedback channel assimilates rare discoveries lossily and cannot manufacture the coverage it would need to keep going. The result is a strong, honest null flanked by two positive findings (search beats training; the correctness wall is a sampling artifact) that together localize the failure and point at the intervention most likely to break it. KernelAscent ships that intervention as an open, automated submission track, so the community can try to turn the null into a curve.

References

- [1] Jason Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [2] Docker engine security. <https://docs.docker.com/engine/security/>, 2025.
- [3] FlashInfer-Bench. <https://flashinfer.ai/2025/10/21/flashinfer-bench.html>, October 2025. Real inference workloads and integration of generated kernels into inference engines.
- [4] FlashInfer trace schema. <https://bench.flashinfer.ai/docs/flashinfer-trace>, 2025. Workload, definition, solution and trace records.
- [5] AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms. <https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>, May 2025.
- [6] Caglar Gulcehre et al. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.
- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [8] Robert E. Kass and Adrian E. Raftery. Bayes factors. *Journal of the American Statistical Association*, 90(430):773–795, 1995. The interpretation thresholds this paper applies to BF01.
- [9] KernelBench-Verified. https://github.com/facebookresearch/kernel_bench_verified, 2025. Stronger input-distribution checks and precision settings over the original kernel evaluation. Its finite hidden tests are not a proof of general correctness.
- [10] Kim et al. Recursive self-improvement through emergent depth. *arXiv preprint arXiv:2608.24735*, August 2026. Cited for positioning; its empirical claims are not independently verified here.
- [11] Recent frontier models are reward hacking. <https://metr.org/blog/2025-06-05-recent-reward-hacking/>, June 2025. Evaluation manipulation motivating independent grading.
- [12] MLPerf inference: Datacenter. <https://mlcommons.org/benchmarks/inference-datacenter/>, 2025.
- [13] NVIDIA compute sanitizer. <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>, 2025.
- [14] SOL-ExecBench. <https://github.com/NVIDIA/SOL-ExecBench>, 2025. Hardware-limit-aware kernel benchmarking.
- [15] Anne Ouyang et al. KernelBench: Can LLMs write efficient GPU kernels? In *International Conference on Machine Learning (ICML)*, 2025. Author list beyond the first author is not recorded in the project spec and must be completed from the proceedings before submission.

- [16] Darwin Gödel machine: AI that improves itself by rewriting its own code. <https://sakana.ai/dgm/>, May 2025. Self-modification and archive-based search.
- [17] Donald J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987.
- [18] John Schulman. Approximating KL divergence. <http://joschu.net/blog/kl-approx.html>, 2020. Source of the k3 estimator used for the trust-region term.
- [19] Self-taught optimizer (STOP): Recursively self-improving code generation. <https://www.microsoft.com/en-us/research/publication/self-taught-optimizer-stop-recursively-self-improving-code-generation/>, 2024.
- [20] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.
- [21] vLLM serving benchmark documentation. <https://docs.vllm.ai/en/stable/cli/bench/serve/>, 2025. A pinned release should be cited rather than the moving stable URL.
- [22] Voyager: An open-ended embodied agent with large language models. <https://voyager.minedojo.org/>, 2023. Executable skill-library accumulation and transfer.
- [23] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [24] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [25] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [26] Zhang et al. HyperAgents. *arXiv preprint arXiv:2603.19461*, 2026. Defines improvement@k and a transfer experiment. Author list beyond the first author is not recorded in the project spec and must be completed before submission.
- [27] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019. The KL-to-reference-policy penalty this paper’s regularized objective adopts.

A What was removed, narrowed, or retracted — and why

Every claim this project has withdrawn or restricted is listed here with the evidence that forced the change. Nothing is dropped silently. A report that shows only its surviving results has removed exactly the information a reader needs in order to calibrate the rest.

A.1 Retracted: claims the data does not support

T1-kernel monotone ordering ($p = 1/120$).

The perfectly ordered five-point decline was an artifact of `KA_EXTRACT=strict`, a candidate filter that discards 66% of 0.5B generations and 5% of 14B generations — a severity that correlates with the independent variable. Under the permissive policy Spearman ρ moves from -1.000 to -0.900 and 3B breaks the order. **The endpoint contrast survives both policies and is retained**; the ordering claim and its p -value are withdrawn.

T1-kernel “14B verifies nothing”.

Recorded twice from a cell that was still running (0 verified at 108 and at 252 attempts). The finished cell has 2 verified in 338. 14B is near-zero, not zero, and the falsifier language now says so.

T3 “frontier self-modification is one-shot”.

The harness requests at most 12 strategies; 5 of 5 interpretable runs sit at 11–12 from round 0 and 4 of 5 never grow. The observed plateau is the cap, not the model. Gains against a frozen procedure remain real and are reported.

T3 cap dose-response (+0.482 at unbounded).

Came from runs carrying the EDQUOT signature ($Q_0 = 0.000$ and four to five all-zero rounds), a filesystem failure in which the grader raises and every candidate scores zero. Clean runs show no dose-response: $-0.046 / +0.021 / -0.009$.

“H100’s torch.compile closed the headroom”.

Offered as the explanation for a spike of scores at exactly 0.50. It is false: the default scorer never touched the compiled baseline at the time. The spike is a plain-torch rewrite landing at parity, which is a different and more consequential fact.

Teacher injection improves compounding.

First reported as +0.100 against +0.051 for the control. It does not survive dropping rounds whose lineage was severed by a resume defect, nor matching on round index, since the arms had unequal valid depth. Matched: control +0.057 [+0.020, +0.094] versus inject +0.075 [+0.035, +0.114]; and on the completed pass-rate board, +0.440 versus +0.445. **Not detected.**

“Retrieval is the worst baseline”.

Stated from one cell at 0.154. The completed four-cell sweep puts retrieval at 0.154, 0.329, 0.385 and 0.426 — a spread of 0.272, against within-cell gaps between any two methods of at most 0.27 and typically half that. It is the worst method in one cell and within 0.013 of the best in another. Retrieval is high variance, not reliably weak, and the original claim was an artifact of reading the low end of that range as the value.

“76% of all scores were exactly 0.50”.

Not reproducible from any artifact. The defensible figure is that 75% of *non-zero* scores fall within ± 0.01 of parity on the 29-task calibration set.

A.2 Narrowed: claims that hold with a smaller scope

The compounding null ($n = 57$ trajectories, $\text{BF}_{01} = 5.6$).

Valid, and narrowed twice. It is a null about acquiring *correctness* rather than optimisation, and specifically about **best-of- k** correctness, which by construction cannot see a reliability gain. The pass-rate board finds a large, family-transferring effect on that axis. These are not contradictory: the null is valid and narrow, and its narrowness was invisible until a metric existed that could see past it.

T1-kernel shape across scale.

Reported as a decline with 14B as the endpoint. The completed 32B sweep recovers to 2.99% from 14B’s 0.00% (Fisher $p = 0.015$), so the curve is a **U** with 14B as its *minimum*. The 0.5B-versus-14B contrast is a contrast with the floor, not with the largest model.

Compile-gain “ $1.34\times$ median”.

Correct and misleading alone. The distribution is bimodal: L1 $15.51\times$, L2 $1.26\times$, L3 $4.53\times$. The median is simply L2’s value because L2 is the majority tier. Reported per tier throughout, because the argument that a plain-torch rewrite scores 0.50 “without compile being strong” holds on L2 and fails on L1.

A.3 Excluded: data that cannot answer the question it was collected for

Severed trajectories (Amendment 5).

`lab_compounding` and `lab_weight_rsi` restored round history on resume but not the trained adapter. Every resumed chunk continued the round numbering while restarting each arm from base weights. The signature is unambiguous: `trainC` collapses at exactly the round named by `resumed_at`, in all 18 resumed cells with no exceptions. All affected rounds are excluded, and the generator refuses to report a cell whose artifact cannot evidence that its weights survived a resume.

Starved trajectories (Amendment 6).

A trajectory whose self arm received fewer than two verified examples per round on average is reported as starved rather than as a measurement of the contrast. At the registered configuration every cell qualifies, so **the registered primary currently reports no number at all**, which is the correct outcome rather than a missing one.

The 32B probe’s first reading.

Its first 12 tasks are L1-heavy and gave 6.94%. Restricting every scale to those same 12 tasks is the valid comparison; the full 29-task sweep gives 2.99%. The partial figure is recorded and not reported.

A.4 Provisional: retained but not load-bearing

Probe-as-intervention ($+0.124 [+0.029, +0.218]$).

$n = 12$ checkpoints, 5–12 tasks per run, against the weakest available comparator (uniform random). Reported in the appendix, not near the abstract.

T4 closed→open.

$n = 1$ per cell against a registered minimum of three seeds, with no interval. Exploratory.

Any A100 speed-scored board.

Measured under a harness that silently rejected every non-PyTorch submission. Retained with the defect stated, never pooled with H100 sets.

A.5 Known confounds carried into the results

- **Cross-rung token budgets differ.** T1 and T2 generate at 2048 tokens; T3 and T5 at 1200. Both resume from per-round checkpoints, so changing them mid-flight would mix budgets inside a single cell, which is worse than a consistent difference. Any cross-rung comparison carries a $1.7\times$ gap and says so.
- **The held set is 5 tasks, not the 20 requested.** A slice does not raise when it runs past the end, so a 25-task pool with 20 taken for training returns whatever remains. At $k = 10$ this caps a round at 50 verified candidates, which bounds every per-round score regardless of its functional form.
- $n = 2$ **trajectories per arm** on the pass-rate board. No interval is quoted on a two-point mean; the per-cell values are given instead.

B Probe-as-intervention, analysed as a secondary result

A linear probe reads “is this kernel correct” out of mid-to-late layers at high pooled AUC. This appendix asks whether that decodable signal converts into a *selection* advantage, and reports it at the right unit with the right denominator.

Three corrections to the earlier analysis. (1) **Unit.** The 19 usable runs are repeated probe-training/test draws over **12 distinct base checkpoints**; draws sharing a checkpoint are not independent, so we cluster on checkpoint. (2) **Denominator.** Each run’s lift is computed over n_{test} tasks, and $n_{\text{test}} \in [5, 12]$ (median 12). A “+0.45 lift” is roughly five problems, so n_{test} is printed in every row. (3) **Estimand.** A high *pooled* correctness AUC can arise because the probe separates easy tasks from hard ones while failing to rank candidates *within* a task — and only within-task ranking can produce a selection gain. Per-candidate probe scores were not retained, so the within-task AUC **cannot be recomputed** from the stored artifacts. We flag this as the binding limitation rather than letting the pooled number stand in for it.

Result. Clustered on base checkpoint, the mean lift over uniform-random selection is $+0.124 [+0.029, +0.218]$ over $n=12$ checkpoints — the 95% CI **excludes zero**: against a uniform-random selector at matched generation budget, probe-guided top-1 selection does lift the correct rate. The result is also *not* TOST-equivalent to zero at the pre-declared $\delta=0.05$, so neither a benefit nor its absence would have been concluded by default — the interval is simply wide. The naive row-level pooling would report $+0.118 [+0.028, +0.209]$ over 19 rows. The probe recovers 36% of the oracle–random headroom on average (95% CI [11%,60%]).

What this does and does not establish. **Uniform random is the weakest possible comparator.** Beating it shows the probe carries *some* usable signal; it does not show the probe is the best use of the same compute, because the informative comparators were not run: mean/length-normalised log-likelihood reranking, a compile-or-run validity filter, and execution agreement. Separately, and importantly, this matched-*generation* selection gain is a different quantity from matched-*verification* harvesting yield, where we measure no advantage. The coherent joint statement is: *extra draws plus a cheap selector find more correct kernels, but the tested probe does not make verification-constrained search economically better.*

The diagnostic. Several runs achieve pooled AUC=1.000 with lifts from +0.000 to +0.250, on training sets as small as 2–5 correct examples. Perfect pooled separation that does not predict selection gain is direct evidence the pooled metric partly measures task difficulty rather than within-task candidate quality — a trap worth flagging for anyone reading correctness probes off a small bank.

Defensible claim. *Decodable correctness information yields a measurable but modest gain over random selection, and does not automatically translate into useful search efficiency under a verification budget.* Settling it needs a rerun on a family-split bank of ≥ 300 independent tasks that logs per-candidate scores, with leave-task-out probe fitting, matched-candidate comparators, task-clustered uncertainty, and a full generation/scoring/verification cost ledger.

Table 15: Probe-as-intervention, one row per run (not per model). n_t = test tasks the row is computed on; head% = fraction of the oracle–random headroom recovered. Rows sharing a base checkpoint are repeated draws and are clustered in the analysis.

run	n_t	K	AUC	random	probe	lift	head%
qwen05k32	12	32	0.831	0.218	0.667	+0.449	64%
qwen05k32b	12	32	0.878	0.247	0.667	+0.420	63%
qwen05_nt24	5	32	0.789	0.086	0.000	-0.086	-75%
Qwen1.5_K32	12	32	0.973	0.040	0.333	+0.293	64%
qwen15k32	11	32	0.961	0.337	0.545	+0.208	43%
qwen15_nt24	5	32	0.806	0.050	0.200	+0.150	100%
qwen14k16	12	16	0.872	0.698	0.750	+0.052	17%
Qwen3	12	16	0.958	0.266	0.667	+0.401	62%
qwen3_nt24	5	32	0.713	0.044	0.200	+0.156	100%
qwen3k32b	12	32	0.817	0.401	0.417	+0.016	3%
qwen3_wt	12	32	0.804	0.299	0.250	-0.049	-11%
qwen7_nt16	12	32	0.824	0.531	0.667	+0.136	62%
qwen7k32	12	32	0.877	0.652	0.750	+0.098	28%
qwen7_nt20	9	32	0.819	0.379	0.111	-0.268	-151%
SmolLM1.7_K32	12	32	1.000	0.083	0.333	+0.250	100%
Yi1.5_K32	10	32	1.000	0.100	0.100	+0.000	–
dsc13k32	9	32	1.000	0.178	0.222	+0.044	28%
DeepSeek6.7B	11	16	0.986	0.121	0.182	+0.061	40%
dsc67k16	12	16	0.826	0.502	0.417	-0.085	-17%